

# ***OPERATING SYSTEM***

## ***UNIT- I***

**Operating Systems Overview:** Introduction, Operating system functions, Operating systems operations, Computing environments, Free and Open-Source Operating Systems

**System Structures:** Operating System Services, User and Operating-System Interface, system calls, Types of System Calls, system programs, Operating system Design and Implementation, Operating system structure, Building and Booting an Operating System, Operating system debugging

### **Overview of Computer Operating Systems**

#### **1. What is an operating system and mention their goals? [SAQ]**

Def: A program that acts as an **interface** between a user of a computer and the computer hardware.

Operating System goals:

- Execute user programs and make solving user problems easier
- Make the computer system convenient to use
- Use the computer hardware in an efficient manner

#### **2. Name the components of Computer System [SAQ]**

*Computer System* can be divided into four components

- Hardware: Provides basic computing resources
- Operating system: Controls and coordinates use of hardware among various applications and users
- Application programs: Define the ways in which the system resources are used to solve the computing problems of the users
- Users: People, machines, other computers

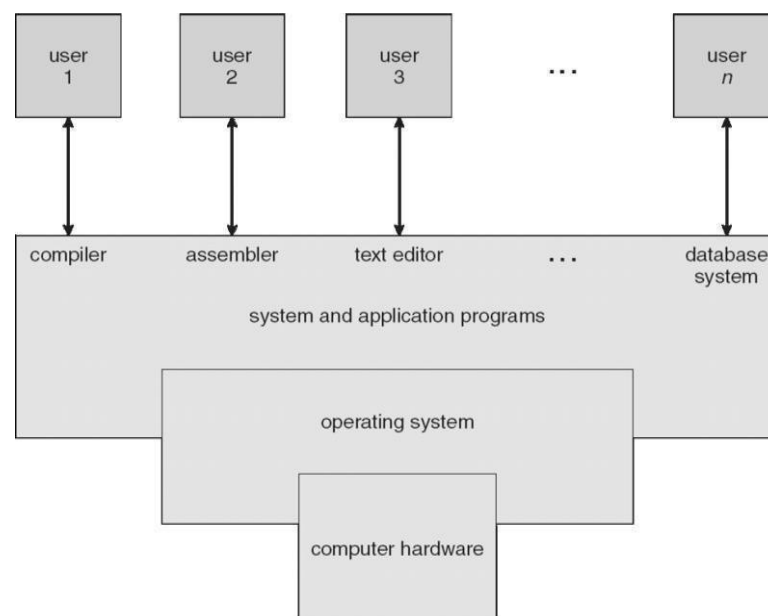


FIG: Abstract view of components of a Computer System

### 3. Explain the Functions of Operating System in Detail [LAQ]

#### **Operating Systems Functions**

Operating Systems provides following functions:

- Process Management
- Memory Management
- Storage Management
  - File System Management
  - Mass Storage Management
  - Caching
  - I/O System

#### **Process Management**

*Process:* A Program under execution is termed as a 'Process'.

A process needs certain resources—including CPU time, memory, files, and I/O devices—to accomplish its task.

A Program is a passive entity whereas a process is an active entity.

Operating system is responsible for the following activities:

- Creating and deleting both user and system processes
- Suspending and resuming processes
- Providing mechanisms for process synchronization
- Providing mechanisms for process communication
- Providing mechanisms for deadlock handling.

#### **Memory Management:**

Main memory is a large array of words or bytes, ranging in size from hundreds of thousands to billions. Each word or byte has its own address.

Operating system is responsible for the following activities:

- Keeping track of which parts of memory are currently being used and by whom
- Deciding which processes and data to move into and out of memory
- Allocating and deallocating memory space as needed

#### **Storage Management:**

Operating system provides a uniform, logical view of information storage.

##### ***(a) File-System Management:***

A file is a collection of related information defined by its creator.

Files represent programs (both source and object forms) and data. Data files may be numeric, alphabetic, alphanumeric, or binary.

Operating system is responsible for the following activities:

- Creating and deleting files
- Creating and deleting directories to organize files
- Supporting primitives for manipulating files and directories
- Mapping files onto secondary storage
- Backing up files on stable (non-volatile) storage media

### ***(b) Mass Storage Management***

Most programs are stored on a disk until loaded into memory and then use the disk as both source and destination of their processing

Because secondary storage is used frequently, it must be used efficiently.

Operating system is responsible for the following activities:

- Free-space management
- Storage allocation
- Disk scheduling

### ***(c) Caching***

Information is normally kept in some storage system (such as main memory).

Upon using, it is copied into a faster storage system called as Cache Memory on a temporary basis.

When we need a particular piece of information, we first check whether it is in the cache.

- If it is, we use the information directly from the cache;
- If it is not, we use the information from the source, putting a copy in the cache under the assumption that we will need it again soon.

| Level                     | 1                                      | 2                             | 3                | 4                | 5                |
|---------------------------|----------------------------------------|-------------------------------|------------------|------------------|------------------|
| Name                      | registers                              | cache                         | main memory      | solid state disk | magnetic disk    |
| Typical size              | < 1 KB                                 | < 16MB                        | < 64GB           | < 1 TB           | < 10 TB          |
| Implementation technology | custom memory with multiple ports CMOS | on-chip or off-chip CMOS SRAM | CMOS SRAM        | flash memory     | magnetic disk    |
| Access time (ns)          | 0.25 - 0.5                             | 0.5 - 25                      | 80 - 250         | 25,000 - 50,000  | 5,000,000        |
| Bandwidth (MB/sec)        | 20,000 - 100,000                       | 5,000 - 10,000                | 1,000 - 5,000    | 500              | 20 - 150         |
| Managed by                | compiler                               | hardware                      | operating system | operating system | operating system |
| Backed by                 | cache                                  | main memory                   | disk             | disk             | disk or tape     |

Table: Performance of Various levels of Storage

### ***(d) I/O Subsystems***

One purpose of OS is to hide peculiarities of hardware devices from the user

I/O subsystem responsible for

- Memory management of I/O including buffering, caching and spooling
- General device-driver interface
- Drivers for specific hardware devices

## Operating-System Operations

### 4. Explain the following operations of Operating System [LAQ]

#### A. Dual-Mode Operation B. Timers

Modern operating systems are interrupt driven.

If there are no processes to execute, no I/O devices to service, and no users to whom to respond, an operating system will sit quietly, waiting for something to happen.

*Events* are almost always signaled by the occurrence of an *Interrupt* or a *Trap*.

*Trap*:(or an Exception) It is a software-generated interrupt caused either by an error or by a specific request from a user program that an operating-system service be performed.

Two Operations

- Dual-Mode Operation
- Timers

#### Dual-Mode Operation

In order to provide the proper execution of the operating system, we must be able to distinguish between the execution of

- Operating-System Code And
- User-defined Code.

We need two separate modes of operation:

- User Mode And
- Kernel Mode (Also Called Supervisor Mode, System Mode, Or Privileged Mode).

A bit, called the mode bit, is added to the hardware of the computer to indicate the current mode:

- Kernel Mode(if Mode Bit is 0) Or
- User Mode(if Mode Bit is 1).

When the computer system is executing on behalf of a user application, the system is in *User Mode*. However, when a user application requests a service from the operating system (via a system call), it must transition from User to Kernel mode to fulfill the request.

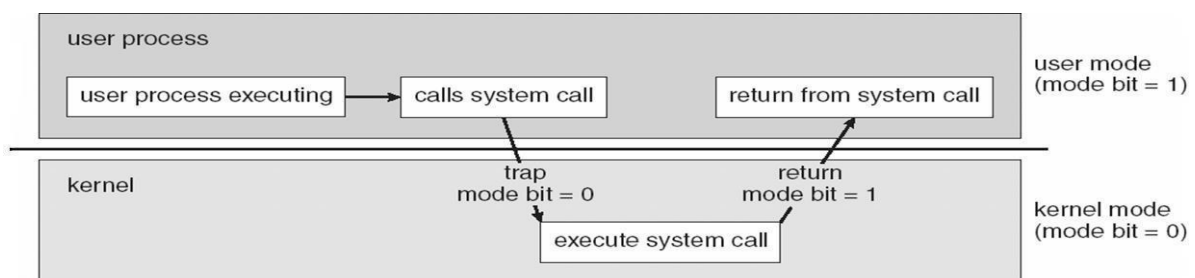


FIG: Transition from User to Kernel Mode

At system boot time, the hardware starts in *Kernel mode*. The operating system is then loaded and starts user applications in *User mode*.

Whenever event or interrupt occurs, the hardware switches from *User mode* to *Kernel mode* (that is, changes the state of the mode bit to 0).

Thus, whenever the Operating System gains control of the computer, it is in *kernel mode*. The system always switches to *user mode* (by setting the mode bit to 1) before passing control to a user program.

## Timers

Def: A timer can be set to interrupt the computer after a specified period.

The period may be

- Fixed Timer or
- Variable Timer

A Fixed Timer is fixed to particular time (for example, 1/60 second)

A Variable Timer is generally implemented by a fixed-rate clock and a counter.(for example, from 1 millisecond to 1 second)

The operating system sets the counter. Every time the clock ticks, the counter is decremented. When the counter reaches 0, an interrupt occurs

## Computing Environments

### 5. Describe various computing environments such as traditional, mobile, distributed, cloud, and real-time systems. [LAQ]

#### Traditional Computing

*Network computers (or thin clients)* are essentially terminals that understand web-based computing. These are used in place of traditional workstations where more security or easier maintenance is desired.

*Mobile computers* can synchronize with PCs to allow very portable use of company information.

*Handheld computers* can synchronize with PCs to allow very portable use of company information.

In the latter half of the previous century, Computing resources were scarce. For a period of time, Systems were either *Batch or Interactive*

*Batch-System* processed job in bulk with predefined input

*Interactive systems* waited for input from users

#### Mobile computing:

Mobile computing refers to computing on handheld smartphones and tablet computers.

These devices share the distinguishing physical features of being portable and lightweight.

Today, mobile systems are used not only for e-mail and web browsing but also for playing music and video, reading digital books, taking photos, and recording high-definition video.

An embedded GPS chip allows a mobile device to use satellites to determine its precise location on earth.

Two operating systems currently dominate mobile computing: *Apple iOS and Google Android*. iOS was designed to run on Apple iPhone and iPad mobile devices. Android powers smartphones and tablet computers available from many manufacturers.

#### Distributed Systems

A distributed system is a collection of physically separate, possibly heterogeneous, computer systems that are networked to provide users with access to the various resources that the system maintains.

*Network*: Communication path between two or more systems.

Networks are characterized based on the distances between their nodes.

*Local-area network (LAN)* connects computers within a room, a building, or a campus.

*Wide-area network (WAN)* usually links buildings, cities, or countries.

A global company may have a WAN to connect its offices worldwide

*Metropolitan-area network (MAN)* could link buildings within a city.

*Personal-area network (PAN)* between a phone and a headset or a smartphone and a desktop computer.

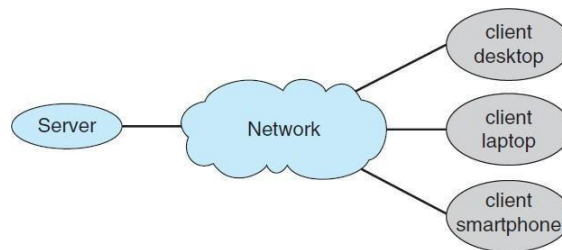
*Network Operating system*: It is an operating system that provides features such as file sharing across the network, along with a communication scheme that allows different processes on different computers to exchange messages.

### Client–Server Computing

Many of today's systems act as *Server systems* to satisfy requests generated by *Client systems*. This form of specialized distributed system, called a *Client–Server system*

Server systems can be broadly categorized as compute servers and file servers:

- *Computer-Server system* provides an interface to which a client can send a request to perform an action (for example, read data). In response, the server executes the action and sends the results to the client.  
A server running a database that responds to client requests for data is an example of such a system.
- *File-server system* provides a file-system interface where clients can create, update, read, and delete files. An example of such a system is a web server that delivers files to clients running web browsers.



**Fig:** General structure of a client–server system.

### Peer-to-Peer Computing

In this model, all nodes within the system are considered peers, and each may act as either a client or a server, depending on whether it is requesting or providing a service.

To participate in a Peer-to-Peer system, a node must first join the network of peers. Once a node has joined the network, it can begin providing services to—and requesting services from—other nodes in the network.

Services available in one of two general ways:

- When a node joins a network, it registers its service with a centralized lookup service on the network. Any node desiring a specific service first contacts this centralized lookup service to determine which node provides the service.
- Peer acting as a client must discover what node provides a desired service by broadcasting a request for the service to all other nodes in the network. The node (or nodes) providing that service responds to the peer making the request.

Figure illustrates such a scenario.

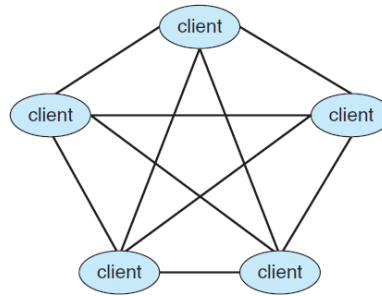


Fig: Peer-to-Peer system with no centralized service.

### Virtualization

Virtualization is a technology that allows operating systems to run as applications within other operating systems.

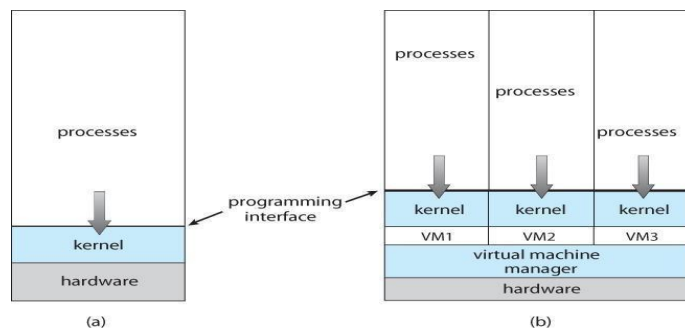


Fig: VMware

*Emulation* is used when the source CPU type is different from the target CPU type.

Windows was the host operating system, and the VMware application was the virtual machine manager VMM. The VMM runs the guest operating systems, manages their resource use, and protects each guest from the others.

### Cloud Computing

Cloud computing is a type of computing that delivers computing, storage, and even applications as a service across a network.

There are actually many types of cloud computing, including the following:

- *Public cloud*—a cloud available via the Internet to anyone willing to pay for the services
- *Private cloud*—a cloud run by a company for that company's own use
- *Hybrid cloud*—a cloud that includes both public and private cloud components
- Software as a service (*SaaS*)—one or more applications (such as word processors or spreadsheets) available via the Internet
- Platform as a service (*PaaS*)—a software stack ready for application use via the Internet (for example, a database server)
- Infrastructure as a service (*IaaS*)—servers or storage available over the Internet (for example, storage available for making backup copies of production data)

These cloud-computing types are not discrete, as a cloud computing environment may provide a combination of several types

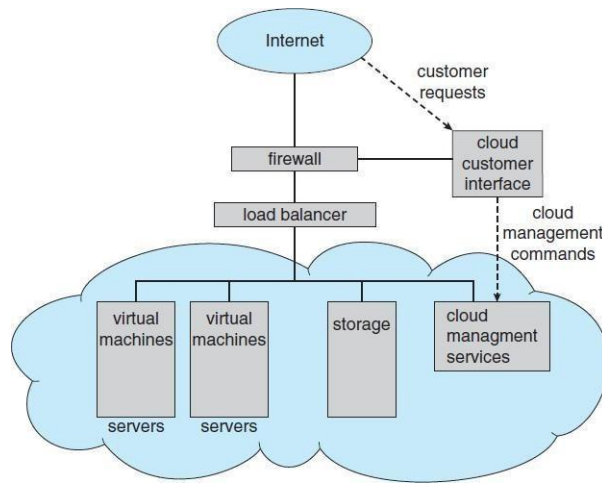


Fig: Cloud Computing

### Real-Time Embedded Systems

These devices are found everywhere, from car engines and manufacturing robots to VCRs and microwave ovens. They tend to have very specific tasks.

Embedded systems almost always run Real-time operating systems

Embedded systems vary considerably. Some are general-purpose computers, running standard operating systems—such as UNIX—with special-purpose applications to implement the functionality.

Embedded systems almost always run *Real-Time Operating Systems*.

### 6. Difference between Client-Server and Peer-to-Peer computing [SAQ]

| Basis For Comparison | Client-Server                                                                                | Peer-To-Peer                                                                                                          |
|----------------------|----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Basic                | There is a specific server and specific clients connected to the server.                     | Clients and server are not distinguished; each node act as client and server.                                         |
| Service              | The client request for service and server respond with the service.                          | Each node can request for services and can also provide the services.                                                 |
| Focus                | Sharing the information.                                                                     | Connectivity.                                                                                                         |
| Data                 | The data is stored in a centralized server.                                                  | Each peer has its own data.                                                                                           |
| Server               | When several clients request for the services simultaneously, a server can get bottlenecked. | As the services are provided by several servers distributed in the peer-to-peer system, a server is not bottlenecked. |
| Expense              | The client-server are expensive to implement.                                                | Peer-to-peer are less expensive to implement.                                                                         |
| Stability            | Client-Server is more stable and scalable.                                                   | Peer-to-Peer suffers if the number of peers increases in the system.                                                  |

## Open Source operating systems

### 7. What is open Source Operating system? Give Example[ SAQ]

*Open-source operating systems* are those available in source-code format rather than as compiled binary code.

Counter to the **copy protection** and **Digital Rights Management(DRM)** movement

Started by **Free Software Foundation (FSF)**, which has “copyleft” **GNU Public License (GPL)**

Linux is the most famous opensource operating system, while Microsoft Windows is a well-known example of the opposite closed-source approach.

#### **LINUX**

As an example of an open-source operating system, consider GNU/Linux.

The GNU project produced many UNIX-compatible tools, including compilers, editors, and utilities.

PCLinuxOS is a LiveCD—an operating system that can be booted and run from a CD-ROM without being installed on a system’s hard disk.

One variant of PCLinuxOS—called “PCLinuxOS Supergamer DVD”—is a LiveDVD that includes graphics drivers and games. A gamer can run it on any compatible system simply by booting from the DVD.

You can run Linux on a Windows system using the following simple, free approach:

1. Download the free “VMware Player” tool from <http://www.vmware.com/download/player/> and install it on your system.
2. Choose a Linux version from among the hundreds of “appliances,” or virtual machine images, available from VMware at <http://www.vmware.com/appliances/>

#### **BSD UNIX**

BSD UNIX has a longer and more complicated history than Linux

Just as with Linux, there are many distributions of BSD UNIX, including FreeBSD, NetBSD, OpenBSD, and DragonflyBSD.

To explore the source code of FreeBSD, simply download the virtual machine image of the version of interest and boot it within VMware, as described above for Linux.

The source code comes with the distribution and is stored in /usr/src/. The kernel source code is in /usr/src/sys.

#### **Solaris**

Solaris is the commercial UNIX-based operating system of Sun Microsystems.

Originally, Sun’s SunOS operating system was based on BSD UNIX. Sun moved to AT&T’s System V UNIX as its base in 1991. In 2005, Sun open-

The source code as it was in 2005 is still available via a source code browser and for download at <http://src.opensolaris.org/source>.

## Open-Source Systems as Learning Tools

### 8. What are open-source systems, and how do they act as learning tools for students and programmers? [SAQ]

The free software movement is driving legions of programmers to create thousands of open-source projects, including operating systems.

Sites like <http://freshmeat.net/> and <http://distrowatch.com/> provide portals to many of these projects. Open-source projects enable students to use source code as a learning tool. They can modify programs and test them help find and fix bugs, and otherwise explore mature, full-featured operating systems, compilers, tools, user interfaces, and other types of programs.

## System Structures

### Operating-System Services

### 9. Explain in detail about the Services provided by Operating System [LAQ]

An operating system provides certain service to the users of those programs

It provides an environment for the execution of programs.

These services provides functions that are helpful to user

- **User interface.** Almost all operating systems have a user interface

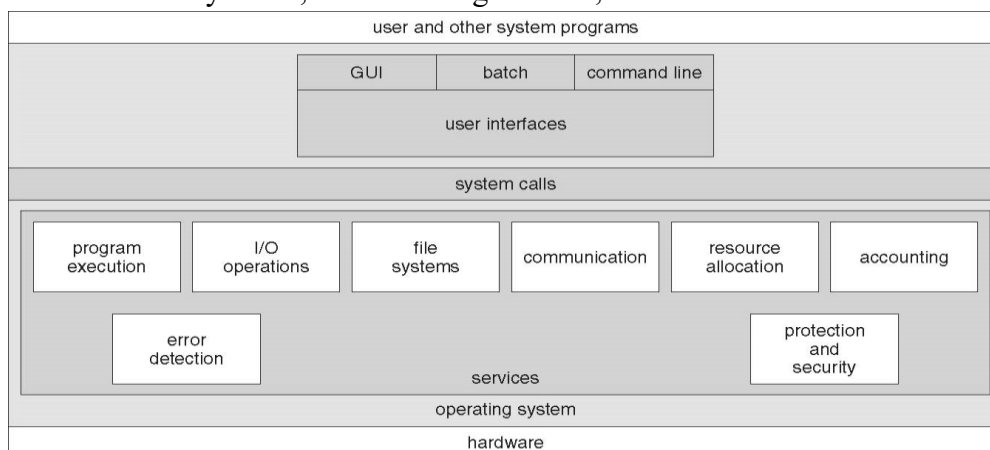
(UI). This interface can take several forms.

- **Command-line interface (CLI):** Uses text commands and a method for entering them
- **Batch interface:** Uses commands and directives to control those commands are entered into files, and those files are executed.
- **Graphical user interface (GUI):** Here, the interface is a window system with a pointing device to direct I/O, choose from menus, and make selections and a keyboard to enter text.

- **Program execution:** The system must be able to load a program into memory and to run that program. The program must be able to end its execution, either normally or abnormally (indicating error).

- **I/O operations:** A running program may require I/O, which may involve a file or an I/O device. For specific devices, special functions may be desired (such as recording to a CD or DVD drive or blanking a CRT screen).

- **File-system manipulation:** Programs need to read and write files and directories. They also need to create and delete them by name, search for a given file, and list file information.



- **Communications.** There are many situations in which one process needs to exchange information with another process. Such communication may occur between processes that are executing on the

same computer or between processes that are executing on different computer systems tied together by a computer network.

- **Error detection:** The operating system needs to be constantly aware of possible errors. Errors may occur in the CPU and memory hardware, in I/O devices and in the user program
- **Resource allocation:** When there are multiple users or multiple jobs running at the same time, resources must be allocated to each of them.
- **Accounting:** We want to keep track of which users use how much and what kinds of computer resources. This record keeping may be used for accounting (so that users can be billed) or simply for accumulating usage statistics
- **Protection and security:** Protection involves ensuring that all access to system resources is controlled. Security starts with requiring each user to authenticate himself or herself to the system, usually by means of a password, to gain access to system resources.

### User and Operating-System Interface

There are several ways for users to interface with the operating system.

Two fundamental approaches.

- Command-Line Interface, or Command Interpreter, that allows users to directly enter commands to be performed by the operating system.
- Allows users to interface with the operating system via a Graphical User Interface, or GUI.

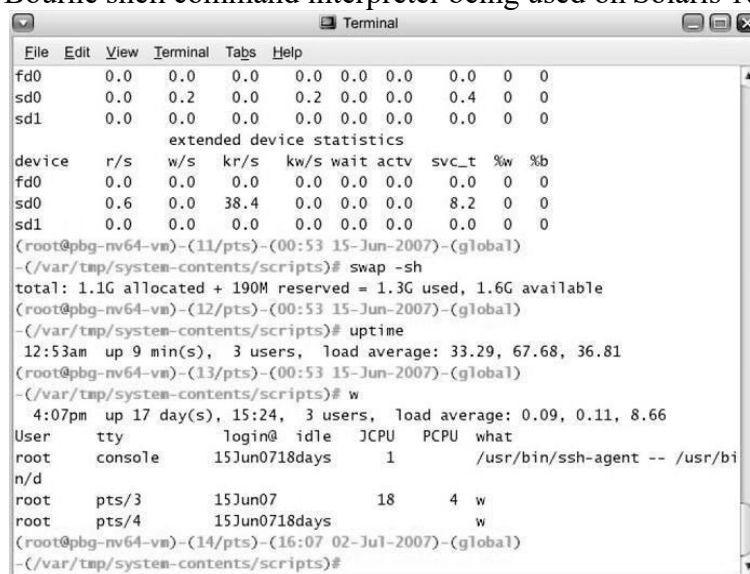
### Command Interpreters

In Windows and UNIX, they treat the command interpreter as a special program that is running when a job is initiated or when a user first logs on (on interactive systems).

On systems with multiple command interpreters to choose from, the interpreters are known as *Shells*.

Ex: On UNIX and Linux systems, a user may choose among several different shells, including the *Bourne shell*, *C shell*, *Bourne-Again shell*, *Korn shell*, and others.

Figure shows the Bourne shell command interpreter being used on Solaris 10.



```
(root@pbg-nv64-vm)-(11/pts)-(00:53 15-Jun-2007)-(global)
-/var/tmp/system-contents/scripts)# swap -sh
total: 1.1G allocated + 190M reserved = 1.3G used, 1.6G available
(root@pbg-nv64-vm)-(12/pts)-(00:53 15-Jun-2007)-(global)
-/var/tmp/system-contents/scripts)# uptime
12:53am up 9 min(s), 3 users, load average: 33.29, 67.68, 36.81
(root@pbg-nv64-vm)-(13/pts)-(00:53 15-Jun-2007)-(global)
-/var/tmp/system-contents/scripts)# w
 4:07pm up 17 day(s), 15:24, 3 users, load average: 0.09, 0.11, 8.66
User   tty          login@   idle   JCPU   PCPU   what
root   console      15Jun07 18days 1      /usr/bin/ssh-agent -- /usr/bi
n/d
root   pts/3        15Jun07 18      4      w
root   pts/4        15Jun07 18days w
(root@pbg-nv64-vm)-(14/pts)-(16:07 02-Jul-2007)-(global)
-/var/tmp/system-contents/scripts)#
```

Fig: The Bourne shell command interpreter in Solaris 10.

The main function of the command interpreter is to get and execute the next user-specified command. Many of the commands given at this level manipulate files: create, delete, list, print, copy, execute, and so on.

Ex: MS-DOS and UNIX

### Graphical-User Interface:

Rather than entering commands directly via a command-line interface, users employ a mouse-based window and menu system characterized by a *Desktop Metaphor*.

The user moves the mouse to position its pointer on images, or icons, on the screen (the desktop) that represent programs, files, directories, and system functions.

Depending on the mouse pointer's location, clicking a button on the mouse can invoke a program, select a file or directory—known as a *Folder*—or pull down menu that contains commands.

Because a mouse is impractical for most mobile systems, smartphones and handheld tablet computers typically use a touch screen interface.



Fig: iPad touch screen

Ex: Pressing and swiping fingers across the screen.

Figure illustrates the touch screen of the Apple iPad.

### Choice of Interface

The choice of whether to use a command-line or GUI interface is mostly one of personal preference.

System administrators who manage computers and power users who have deep knowledge of a system frequently use the command-line interface.

### System calls

#### 10. Explain system calls and describe different methods of passing parameters. [LAQ]

Provides an interface to the services that are made available by the Operating Systems

Ex: To illustrate how system calls are used: writing a simple program to read data from one file and copy them to another file.

The first input that the program will need is the names of the two files: the input file and the output file. In an interactive system, this approach will require a sequence of system calls, first to write a prompting message on the screen and then to read from the keyboard the characters that define the two files.

This system call sequence is shown in Figure

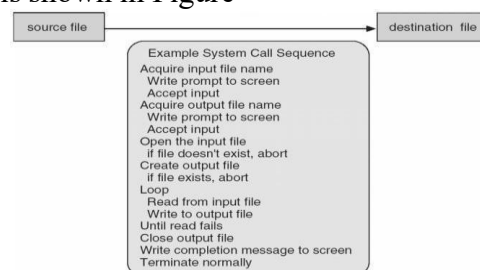


FIG: Example of how System Calls are used

The relationship between an API, the system-call interface, and the operating system is shown in following Figure. It illustrates how the operating system handles a user application invoking the `open()` system call.

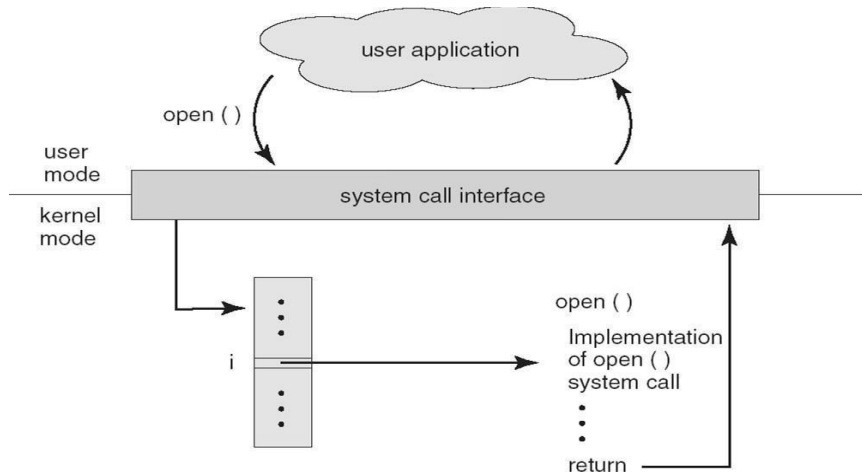


FIG: Handling of a User Application invoking the open() System Call

### System Call Parameter Passing

Three general methods used to pass parameters to the OS

- Pass the parameters in *registers*. In some cases, may be more parameters than registers.
- Parameters stored in a *block*, or table, in memory, and address of block passed as a parameter in a register
- Parameters placed, or *pushed*, onto the *stack* by the program and *popped* off the stack by the operating system

Block and stack methods do not limit the number or length of parameters being passed

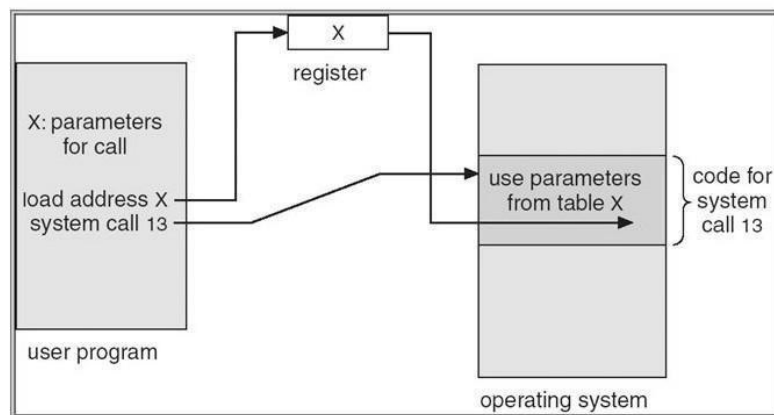


FIG: Passing of Parameters as a Table

### Types of System Calls:

#### 11. Explain the various types of System Calls [LAQ]

System calls can be grouped into five major categories:

- Process Control,
- File Manipulation,
- Device Manipulation,
- Information Maintenance and
- Communications.

#### Process control

A running program needs to be able to halt its execution either normally (end) or abnormally (abort).

If a system call is made to terminate the currently running program abnormally, or if the program

runs into a problem and causes an error trap, a dump of memory is taken and an error message generated. The dump is written to disk and may be examined by a debugger

*Debugger* is a system program designed to aid the programmer in finding and correcting bugs and to determine the cause of the problem.

A *Process* or *Job* executing one program may want to load and execute another program. This feature allows the command interpreter to execute a program as directed

Ex: A user command, the click of a mouse, or a batch command.

System calls in case of Process Management are:

- end, abort
- load, execute
- create process, terminate process
- get process attributes, set process attributes
- wait for time
- wait event, signal event
- allocate and free memory

Ex: Single-tasking system and multitasking system

MS-DOS operating system is an example of a single-tasking system.

It has a command interpreter that is invoked when the computer is started.

Because MS-DOS is single-tasking, it uses a simple method to run a program and does not create a new process.

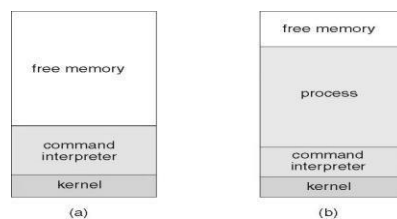


FIG: MS-DOS Execution (a) At System Startup (b) Running a program

FreeBSD is an example of a multitasking system. When a user logs on to the system, the shell of the user's choice is run. Since FreeBSD is a multitasking system, the command interpreter may continue running while another program is executed

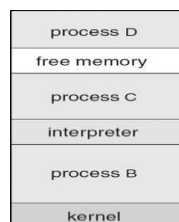


FIG: FreeBSD running Multiple Program

## File management

Firstly, we need to be able to create and delete files.

Either system call requires the name of the file and some of the file's attributes. Once the file is created, we need to open it and to use it. We may also read, write, or reposition (rewinding or skipping to the end of the file).

Finally, we need to close the file, indicating that we are no longer using it.

File attributes include the file name, a file type, protection codes, accounting information, and so on. At least two system calls, get file attribute and set file attribute, are required for this function.

System calls in case of File Management are:

- create file, delete file
- open, close
- read, write, reposition
- get file attributes, set file attributes

### **Device management**

A process may need several resources to execute—main memory, disk drives, access to files, and so on. If the resources are available, they can be granted, and control can be returned to the user process. Otherwise, the process will have to wait until sufficient resources are available.

If there are multiple users of the system, the system may require us to first request the device, to ensure exclusive use of it. After we are finished with the device, we release it.

System calls in case of Device Management are:

- request device, release device
- read, write, reposition
- get device attributes, set device attributes
- logically attach or detach devices

### **Information maintenance**

Many system calls exist simply for the purpose of transferring information between the user program and the operating system.

Ex: Most systems have a system call to return the current time and date .

Other systemcalls may return information about the system, such as the number of current users, the version number of the operating system, the amount of free memory or disk space, and so on.

System calls in case of Information maintenance are:

- get time or date, set time or date
- get system data, set system data
- get process, file, or device attributes
- set process, file, or device attributes

### **Communications**

Two common models of Interprocess communication:

1. Message-Passing model and
2. Shared-memory model.

In the Message-Passing model, the communicating processes exchange messages with one another to transfer information. Messages can be exchanged between the processes either directly or indirectly through a common mailbox. Before communication can take place, a connection must be opened.

System calls in case of Communications are:

- create, delete communication connection
- send, receive messages
- transfer status information

- attach or detach remote devices

### **Systems Programs**

#### **12. Explain different types of system programs and their role in OS functioning. [LAQ]**

Another aspect of a modern system is its collection of system programs.

*System programs*, also known as system utilities, provide a convenient environment for program development and execution.

They can be divided into these categories:

- **File management:** These programs create, delete, copy, rename, print, dump, list, and generally manipulate files and directories.
- **Status information:** Some programs simply ask the system for the date, time, amount of available memory or disk space, number of users, or similar status information.
- **File modification:** Several text editors may be available to create and modify the content of files stored on disk or other storage devices.
- **Programming-language support:** Compilers, assemblers, debuggers, and interpreters for common programming languages are often provided with the operating system or available as a separate download.
- **Program loading and execution:** Once a program is assembled or compiled, it must be loaded into memory to be executed. The system may provide absolute loaders, relocatable loaders, linkage editors, and overlay loaders.
- **Communications:** These programs provide the mechanism for creating virtual connections among processes, users, and computer systems. They allow users to send messages to one another's screens, to browse Web pages, to send e-mail messages etc
- **Background services:** All general-purpose systems have methods for launching certain system-program processes at boot time.

Along with system programs, most operating systems are supplied with programs that are useful in solving common problems or performing common operations.

### **Operating Systems Structure**

#### **13. Describe Operating System structures: Simple, Layered, Microkernel, Modules, and Hybrid.**

A system as large and complex must be engineered carefully. To partition the task into small components rather than have one monolithic {single components} system.

These components are interconnected and melded into Kernel

- Simple Structure
- Layered Approach
- Micro-Kernel
- Modules

#### **Simple Structure**

Many commercial systems do not have well-defined structures.

Such operating systems started as small, simple, and limited systems

Ex 1: MS-DOS.

Main functionality is the least space, so it was not divided into modules carefully. Its interfaces and levels of Functionality are not well separated

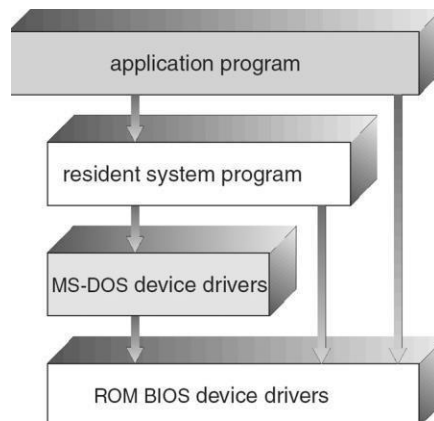


Fig:MS-DOS Layer Structure

Ex 2: UNIX operating system.

UNIX is another system that initially was limited by hardware functionality.

It consists of two separable parts:

- Kernel and the
- System programs.

Kernel is further separated into a series of interfaces and device drivers. Kernel provides the file system, CPU scheduling, Memory management and other Operating System functions through System calls

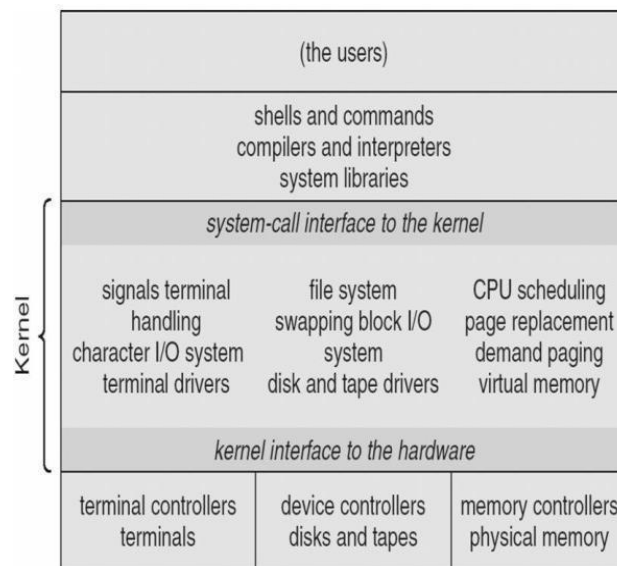


FIG: UNIX System Structure

## Layered Approach

The operating system is divided into a number of layers (levels)

Each layer is built on top of lower layers.

Bottom layer (layer 0), is the hardware; the highest (layer N) is the user interface.

With modularity, layers are selected such that each uses functions (operations) and services of only lower-level layers

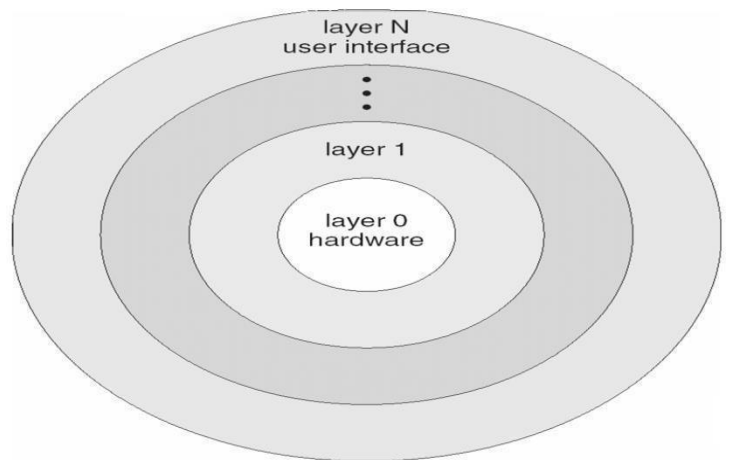


FIG: Layered Operating System

Advantages: Simplicity of construction and debugging.

First layer can be debugged without any concern for the rest of the system. Once the first layer is debugged, its correct functioning can be assumed while the second layer is debugged, and so on. If an error is found during the debugging of a particular layer, the error must be on that layer, because the layers below it are already debugged.

Disadvantages:

- Defining the various layers is done appropriately. Because a layer can use only lower-level layers, careful planning is necessary.
- They tend to be less efficient than other types.

## Microkernel System Structure

An operating system called Mach that modularized the kernel using the Microkernel approach.

These provide minimal process and memory management

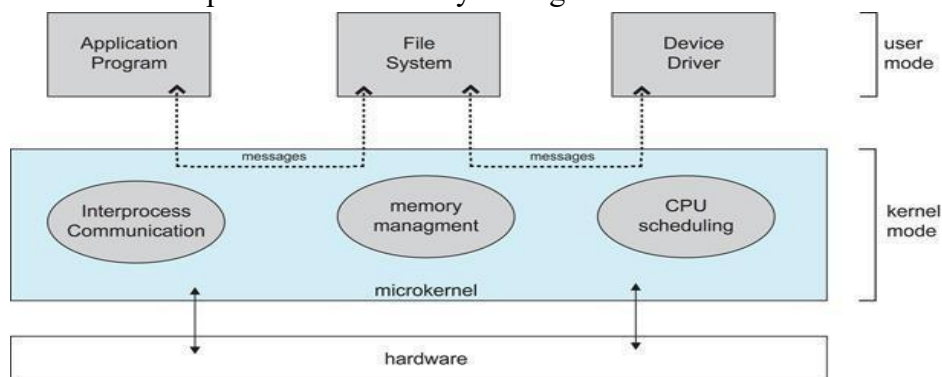


Fig: Architecture of a typical Microkernel

Main function of the microkernel is to provide a communication facility between the client program and the various services that are also running in user space.

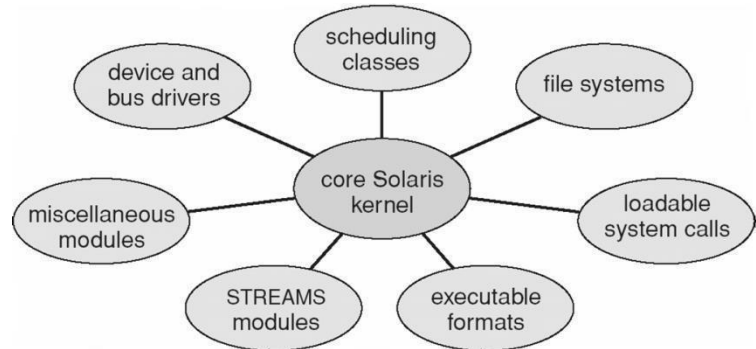
Advantages:

- Easier to extend a microkernel
- Easier to port the operating system to new architectures
- More reliable (less code is running in kernel mode)
- More secure

*Disadvantage:* Performance overhead of user space to kernel space communication

## Modules

FIG: Solaris Loadable Modules



Kernel has a set of core components and dynamically links in additional services either during boot time or during run time.

Such a strategy uses dynamically loadable modules and is common in modern implementations of UNIX, such as Solaris, Linux, and Mac OS X.

## Hybrid Systems

Three hybrid systems: the Apple Mac OS X operating system and the two most prominent mobile operating systems—iOS and Android.

### Mac OS X Structure

The Apple Macintosh Mac OS X operating system uses a hybrid structure.

The top layers include the *Aqua* user interface and a set of application environments and services.

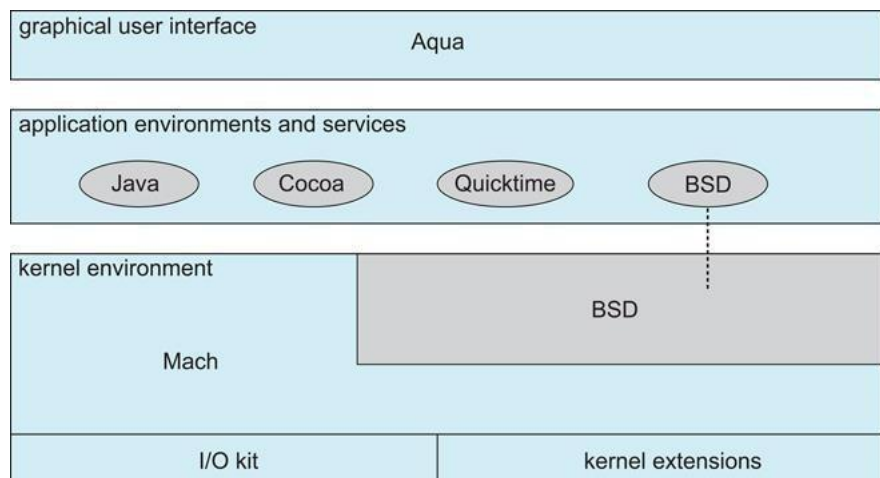


FIG: MAC OS X Structure

*Cocoa* environment specifies an API for the Objective-C programming language, which is used for writing Mac OS X applications.

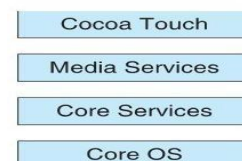
Below these layers is the *kernel environment*, which consists primarily of the Mach microkernel and the BSD UNIX kernel.

Mach provides memory management; support for remote procedure calls (RPCs) and interprocess communication (IPC) facilities, including message passing; and thread scheduling.

### iOS

iOS is a mobile operating system designed by Apple to run its smartphone, the *iPhone* and its tablet computer, the *iPad*.

iOS is structured on the Mac OS X operating system, with added functionality pertinent to mobile devices, but does not directly run



Mac OS X applications.

The structure of iOS appears in Figure

Fig: Architecture of Apple's iOS

Cocoa Touch is an API for Objective-C that provides several frameworks for developing applications that run on iOS devices.

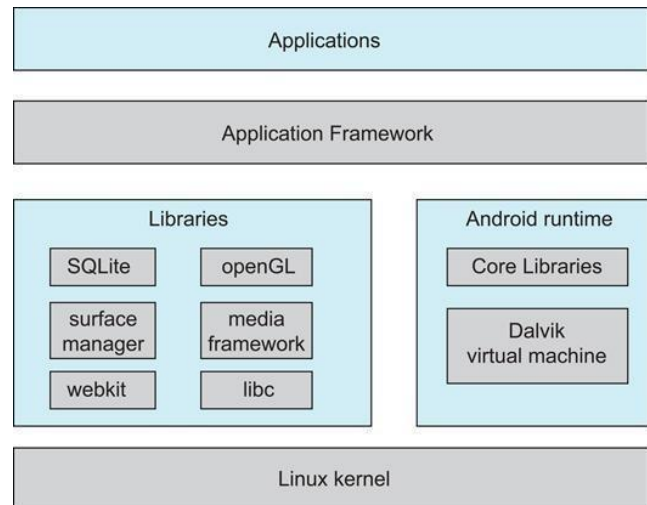
The media services layer provides services for graphics, audio, and video. The core services layer provides a variety of features, including support for cloud computing and databases. The bottom layer represents the core operating system, which is based on the kernel environment

### Android

The Android operating system was designed by the Open Handset Alliance (led primarily by Google) and was developed for Android smartphones and tablet computers.

Android runs on a variety of mobile platforms and is open-sourced

The structure of Android appears in Figure  
Fig:Architecture of Google's Android



It is similar to iOS in that it is a layered stack of software that provides a rich set of frameworks for developing mobile applications.

At the bottom of this software stack is the Linux kernel. Linux is used primarily for process, memory, and device-driver support for hardware and has been expanded to include power management.

## Design and Implementation in Operating System

14. Write a detailed note on OS design considerations, including goals, mechanism–policy separation, and architectural models.[LAQ]

### Design Goals of Operating System

Design goals are the strategic objectives of OS design.

These goals define *what the OS should achieve* and act as guidelines to evaluate whether the OS is meeting expectations and user needs.

### Key Design Goals

| Goal        | Meaning                                                            |
|-------------|--------------------------------------------------------------------|
| Reliability | OS should work without failure, should recover quickly from errors |
| Efficiency  | Better utilization of CPU, Memory, I/O, Disk etc                   |
| Robustness  | System should handle abnormal conditions safely                    |
| Flexibility | Support multiple architectures, new functions and upgrades easily  |

| Goal                  | Meaning                                                         |
|-----------------------|-----------------------------------------------------------------|
| Security & Protection | Protect data, processes, files, memory from unauthorized access |
| Performance           | Low response time, high throughput, low latency                 |
| User Convenience      | Easy to use environment for normal users and programmers        |
| Portability           | OS code should run on different hardware with minimal changes   |
| Scalability           | Must support small system to large multi processor system       |

### Goal Driven Design:

Designers identify all possible goals to check compatibility by removing conflicts and prioritize using **Cost Benefit Analysis (CBA)**.

CBA includes:

- cost impact
- development time
- effect on user experience
- effect on other subsystems

Thus final OS decisions are always user and stakeholder friendly.

### Mechanisms and Policies

Operating System consists of two parts:

| Concept   | Definition                                    | Purpose                    |
|-----------|-----------------------------------------------|----------------------------|
| Mechanism | How to do something (implementation strategy) | Provide capability         |
| Policy    | What decision to make and when                | Decide usage of capability |

### Example:

- Mechanism: context switching
- Policy: when to perform context switching (preemptive or non preemptive)
- Mechanism: semaphore / lock variable
- Policy: what rule to apply for critical section scheduling (priority, fairness etc.)

### Conclusion:

Mechanisms are tools. Policies are decision rules. A good OS must cleanly separate mechanism and policy → so mechanisms can be reused with different policies.

### Implementation of Operating System

Implementation is the process of writing OS code in high level language → compiling → generating executable kernel.

## Responsibilities of OS Implementation

- Program execution control
- Memory management (allocation, deallocation, protection)
- Interrupt handling
- File system management
- CPU scheduling
- Device / I/O management
- Protection and security
- System call implementation

## Modern OS characteristics:

- modular architecture
- layered / microkernel / monolithic hybrid
- stored in ROM / Flash → bootstraps at power-on
- supports multi-core / virtualization / distributed environment

## Types of Operating Systems

| Type         | Examples                           | Purpose                                         |
|--------------|------------------------------------|-------------------------------------------------|
| GUI Based    | Windows, MacOS                     | User friendly graphical interaction             |
| Command Line | Linux, UNIX                        | Developer oriented, scripting powerful          |
| Real Time OS | VxWorks, QNX                       | Industrial control / robotics / medical systems |
| Embedded OS  | Android (embedded version), WearOS | IoT devices, smart appliances, controllers      |

Operating System is the most important system software which manages **hardware resources**, controls **application execution**, provides **services**, ensures **security** and guarantees **efficient performance**. Good OS design depends on correct identification of goals, correct separation of mechanisms and policies, and proper implementation architecture.

## Operating System Structure

15. Explain the operating system structure with emphasis on multiprogramming and time-sharing.[LAQ]

An operating system provides the environment within which programs are executed. One of

the most important aspects of operating systems is the ability to multiprogram.

*Multiprogramming* increases CPU utilization by organizing jobs (code and data) so that the CPU always has one to execute.

The idea is as follows:

The operating system keeps several jobs in memory simultaneously (Fig).

Since main memory is too small to accommodate all jobs, the jobs are kept initially on the disk in the *Job Pool*. This pool consists of all processes residing on disk awaiting allocation of main memory.

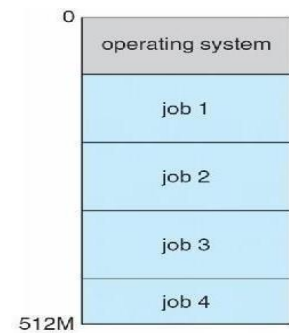


Fig: Memory layout for a multiprogramming system

In a non-multiprogrammed system, the CPU would sit idle.

In a multiprogrammed system, the operating system simply switches to, and executes, another job.

*Time sharing (or multitasking)* is a logical extension of multiprogramming. In time-sharing systems, the CPU executes multiple jobs by switching among them, but the switches occur so frequently that the users can interact with each program while it is running.

Time sharing requires an interactive computer system, which provides direct communication between the user and the system.

### Operating-System Debugging

#### **16. Explain system debugging, failure analysis, performance tuning, and the role of DTrace[LAQ]**

Debugging is the activity of finding and fixing errors in a system, both in hardware and in software.

Performance problems are considered bugs, so debugging can also include performance tuning, which seeks to improve performance by removing processing bottlenecks.

#### **Failure Analysis**

If a process fails, most operating systems write the error information to a log file to alert system operators or users that the problem occurred.

The operating system can also take a core dump—a capture of the memory of the process—and store it in a file for later analysis.

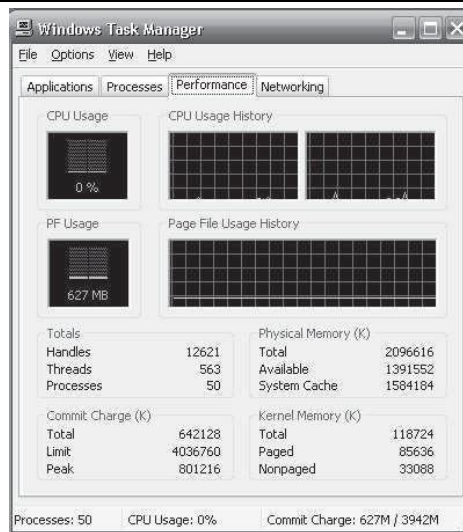
A failure in the kernel is called a *Crash*. When a crash occurs, error information is saved to a log file, and the memory state is saved to a *Crash Dump*.

#### **Performance Tuning**

Performance tuning seeks to improve performance by removing processing bottlenecks.

To identify bottlenecks, we must be able to monitor system performance. Thus, the operating system must have some means of computing and displaying measures of system behavior.

In a number of systems, the operating system does this by producing *trace listings* of system behavior



## DTrace

DTrace is a facility that dynamically adds probes to a running system, both in user processes and in the kernel.

Ex: Figure follows an application as it executes a system call (`ioctl()`) and shows the functional calls within the kernel as they execute to perform the system call.

Lines ending with “U” are executed in user mode, and lines ending in “K” in kernel mode

```
# ./all.d 'pgrep xclock' XEventsQueued
dtrace: script './all.d' matched 52377 probes
CPU FUNCTION
0 -> XEventsQueued U
0 -> _XEventsQueued U
0 -> _X11TransBytesReadable U
0 <- _X11TransBytesReadable U
0 -> _X11TransSocketBytesReadable U
0 <- _X11TransSocketBytesreadable U
0 -> ioctl U
0 -> ioctl K
0 -> getf K
0 -> set_active_fd K
0 <- set_active_fd K
0 <- getf K
0 -> get_umatamodel K
0 <- get_umatamodel K
...
0 -> releasef K
0 -> clear_active_fd K
0 <- clear_active_fd K
0 -> cv_broadcast K
0 <- cv_broadcast K
0 <- releasef K
0 <- ioctl K
0 <- ioctl U
0 <- _XEventsQueued U
0 <- XEventsQueued U
```

Fig: Solaris 10 dtrace follows a system call within the kernel.

The DTrace tool meets these requirements and provides a dynamic, safe, low-impact debugging environment.

## ***System Boot***

### **17. Describe the complete system boot process and the role of the bootstrap loader.[LAQ]**

After an operating system is generated, it must be made available for use by the hardware.

The procedure of starting a computer by loading the kernel is known as *booting the system*.

On most computer systems, a small piece of code known as the *bootstrap program* or *bootstrap loader* locates the kernel, loads it into main memory, and starts its execution.

The bootstrap program can perform a variety of tasks.

- Run diagnostics to determine the state of the machine. If the diagnostics pass, the program can continue with the booting steps.
- It can also initialize all aspects of the system, from CPU registers to device controllers and the contents of main memory.

Some systems—such as cellular phones, tablets, and game consoles—store the entire operating system in ROM. Storing the operating system in ROM is suitable for small operating systems, simple supporting hardware, and rugged operation. A problem with this approach is that changing the bootstrap code requires changing the ROM hardware chips.

Some systems resolve this problem by using erasable programmable read-only memory (EPROM), which is read only except when explicitly given a command to become writable. All forms of ROM are also known as firmware, since their characteristics fall somewhere between those of hardware and those of software.

### **ShortAnswerQuestions**

1. Define Operating System.
2. What are the main goals of an Operating System?
3. What is a Process?
4. Define Multiprogramming.
5. What is Time-Sharing?
6. What is a Trap or Exception?
7. What is Dual-Mode Operation?
8. Define System Call.
9. What is a Shell?
10. What is a Graphical User Interface (GUI)?
11. What is a Microkernel?
12. Define Virtualization.
13. What is a Distributed System?
14. What is Booting?
15. What is Interprocess Communication (IPC)?